

LAPORAN PRAKTIKUM TUGAS
MATERI 4 :
CPU Scheduling (FCFS, SJF, dan Round Robin)



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
MUHAMMAD RIDHO 2410651009 (A)

Tanggal : 3-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

- Tujuan Praktikum

Praktikum ini bertujuan untuk:

1. Menganalisis perbedaan hasil eksekusi antara tiga algoritma penjadwalan CPU: FCFS, SJF, dan Round Robin.
2. Mengukur dan membandingkan Average Waiting Time (AWT) serta Turnaround Time dari setiap algoritma.
3. Memahami kelebihan, kekurangan, serta kondisi penggunaan masing-masing algoritma dalam sistem operasi nyata.

- Dasar Teori

Penjadwalan proses merupakan mekanisme yang digunakan sistem operasi untuk menentukan proses mana yang akan dieksekusi oleh CPU pada suatu waktu tertentu.

Tiga algoritma utama yang digunakan dalam praktikum ini adalah:

- **FCFS (First Come, First Served):**
Proses dieksekusi berdasarkan urutan kedatangan. Sederhana, namun rentan terhadap *convoy effect*.
- **SJF (Shortest Job First):**
Memilih proses dengan waktu eksekusi (burst time) paling pendek terlebih dahulu.
SJF memberikan rata-rata waktu tunggu paling kecil tetapi bisa menyebabkan *starvation* untuk proses panjang.
- **Round Robin (RR):**
Memberikan waktu eksekusi CPU (quantum) secara bergiliran ke setiap proses.
Cocok untuk sistem multitasking atau interaktif karena menjamin keadilan (*fairness*).

- Analisis dan Pembahasan

A. Analisis Eksperimen Algoritma

1. Algoritma dengan AWT terkecil:

SJF memberikan **Average Waiting Time (AWT)** terkecil karena menyelesaikan proses dengan burst time paling pendek terlebih dahulu.

2. Perbedaan hasil antar algoritma:

Setiap algoritma memiliki kebijakan berbeda dalam menentukan urutan eksekusi proses.

- FCFS: berdasarkan waktu kedatangan.
- SJF: berdasarkan burst time terkecil.
- RR: berdasarkan giliran waktu (quantum).

3. Kapan menggunakan masing-masing algoritma:

- **FCFS**: digunakan pada sistem batch processing atau job queue yang tidak memerlukan respons cepat.
- **SJF**: digunakan jika durasi eksekusi proses diketahui dan ingin efisiensi waktu tunggu terbaik.
- **Round Robin**: digunakan untuk sistem interaktif dan multitasking, agar setiap proses mendapat kesempatan eksekusi merata.

B. Analisis Performa

- **FCFS** bekerja baik jika burst time proses relatif sama atau proses pendek datang lebih dulu.
- **SJF** hampir selalu menghasilkan **AWT paling kecil**, karena strategi optimalnya memprioritaskan proses dengan waktu eksekusi terpendek.
- **Round Robin** seimbang antara efisiensi dan keadilan, tergantung pada pemilihan nilai **quantum**.

C. Context Switching

- **Round Robin** menghasilkan **context switch paling banyak**, karena pergantian proses terjadi setiap kali quantum habis.
- **Quantum kecil** → **context switch tinggi** → **overhead besar**.
- **Quantum besar** → **lebih sedikit context switch**, tapi **respons lebih lambat**.
- Nilai **quantum sedang (8)** sering menjadi titik seimbang antara respons cepat dan efisiensi.

D. Fairness dan Starvation

- **Round Robin** paling adil karena semua proses mendapat giliran CPU.
- **FCFS** merugikan proses pendek yang datang setelah proses besar.
- **SJF** berpotensi menyebabkan *starvation* untuk proses besar jika proses kecil terus datang.
Solusi: gunakan **Aging**, yaitu menaikkan prioritas proses yang terlalu lama menunggu.

TUGAS 1

Data set 1:

```
root@RIDHO:~# nano fcfs.c
root@RIDHO:~# gcc fcfs.c -o fcfs
root@RIDHO:~# ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      10      10      10        0
P2        1       5      15      14        9
P3        2       8      23      21       13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23
```

```
root@RIDHO:~# ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4
```

```
Proses P1
  Arrival Time: 0
  Burst Time: 5
```

```
Proses P2
  Arrival Time: 1
  Burst Time: 3
```

```
Proses P3
  Arrival Time: 2
  Burst Time: 8
```

```
Proses P4
  Arrival Time: 3
  Burst Time: 6
```

```
=== HASIL PENJADWALAN FCFS ===
```

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

```
Average Turnaround Time: 11.25
Average Waiting Time: 5.75
```

```
=== GANTT CHART ===
```

```
| P1 | P2 | P3 | P4 |
0    5    8   16   22
```

Data set 2

```
root@RIDH0:~# nano sjf.c
root@RIDH0:~# gcc sjf.c -o sjf
root@RIDH0:~# ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3
```

```
Proses P1
  Arrival Time: 0
  Burst Time: 10
```

```
Proses P2
  Arrival Time: 1
  Burst Time: 5
```

```
Proses P3
  Arrival Time: 2
  Burst Time: 8
```

```
=== HASIL PENJADWALAN SJF ===
```

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13

```
Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

```
=== URUTAN EKSEKUSI ===
```

```
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

```

root@RIDHO:~# ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN SJF ===

```

PID	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	22	20	12
P4	3	6	14	11	5

```

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3

```


TUGAS 2

2. Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban :

1. Algoritma SJF (Shortest Job First) memberikan Average Waiting Time terkecil.

Baik pada Data Set 1 maupun Data Set 2, hasil rata-rata waktu tunggu selalu paling kecil dibanding FCFS dan Round Robin.

2. Karena setiap algoritma memiliki kebijakan penjadwalan yang berbeda dalam menentukan urutan eksekusi proses.

3. Gunakan FCFS (First Come First Served) ketika:

- Sistem memiliki beban proses besar dan datang berurutan, seperti batch processing atau job queue.
- Tidak masalah jika ada perbedaan besar antara waktu eksekusi proses.
- Tujuannya adalah kesederhanaan dan fairness berdasarkan urutan kedatangan.

4. Gunakan SJF (Shortest Job First) ketika:

- Sistem mengetahui atau dapat memperkirakan burst time (durasi eksekusi) setiap proses.
- Tujuannya adalah efisiensi maksimum — meminimalkan waiting time dan turnaround time.
- Proses kecil sering datang lebih banyak daripada proses besar.

5. Gunakan Round Robin (RR) ketika:

- Sistem bersifat interaktif atau multitasking, di mana setiap proses harus mendapatkan waktu CPU secara bergantian.
- Tujuannya adalah keadilan (fairness) antar proses dan respons cepat terhadap input Pengguna

Tugas 3

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    bool is_completed;
};

int main() {
    FILE *fin=fopen("input_sjf.txt","r");
    FILE *fout=fopen("hasil_sjf.txt","w");
    if(fin==NULL){printf("File input_sjf.txt tidak ditemukan.\n");return 1;}
    int n;
    fscanf(fin,"%d",&n);
    struct Process proc[n];
    for(int i=0;i<n;i++){
        fscanf(fin,"%d %d",&proc[i].arrival_time,&proc[i].burst_time);
        proc[i].pid=i+1;
        proc[i].is_completed=false;
    }
    fclose(fin);

    int current_time=0,completed=0,sequence[n];
    while(completed<n){
        int shortest_job=-1,min_burst=99999;
        for(int i=0;i<n;i++){
            if(!proc[i].is_completed&&proc[i].arrival_time<=current_time&&proc[i].burst_time<min_burst){
                min_burst=proc[i].burst_time;
                shortest_job=i;
            }
        }
        if(shortest_job==-1){current_time++;continue;}
    }
```

```

        if(shortest_job==1){current_time++;continue;}
        sequence[completed]=shortest_job;
        current_time+=proc[shortest_job].burst_time;
        proc[shortest_job].completion_time=current_time;
        proc[shortest_job].turnaround_time=proc[shortest_job].completion_time-proc[shortest_job].arrival_time;
        proc[shortest_job].waiting_time=proc[shortest_job].turnaround_time-proc[shortest_job].burst_time;
        proc[shortest_job].is_completed=true;
        completed++;
    }

    printf("\n=== HASIL PENJADWALAN SJF ===\n");
    fprintf(fout,"=== HASIL PENJADWALAN SJF ===\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(fout,"PID\tAT\tBT\tCT\tTAT\tWT\n");
    float total_tat=0,total_wt=0;
    for(int i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",proc[i].pid,proc[i].arrival_time,proc[i].burst_time,proc[i].completion_time,proc[i].turnaround_time,proc[i].waiting_time);
        fprintf(fout,"P%d\t%d\t%d\t%d\t%d\t%d\n",proc[i].pid,proc[i].arrival_time,proc[i].burst_time,proc[i].completion_time,proc[i].turnaround_time,proc[i].waiting_time);
        total_tat+=proc[i].turnaround_time;
        total_wt+=proc[i].waiting_time;
    }
    float avg_tat=total_tat/n,avg_wt=total_wt/n;
    printf("\nAverage Turnaround Time: %.2f\n",avg_tat);
    printf("Average Waiting Time: %.2f\n",avg_wt);
    fprintf(fout,"Average Turnaround Time: %.2f\n",avg_tat);
    fprintf(fout,"Average Waiting Time: %.2f\n",avg_wt);

    printf("\n=== URUTAN EKSEKUSI ===\nProses dieksekusi dalam urutan: ");
    fprintf(fout,"n=== URUTAN EKSEKUSI ===\nProses dieksekusi dalam urutan: ");
    for(int i=0;i<n;i++){
        printf("P%d",proc[sequence[i]].pid);
        fprintf(fout,"P%d",proc[sequence[i]].pid);
        if(i<n-1){printf(" -> ");fprintf(fout," -> ");}
    }
    printf("\n\n=== GANTT CHART ===\n");
    fprintf(fout,"n\n=== GANTT CHART ===\n");
    int time_line=0;

```

```

    for(int i=0;i<n;i++){
        printf(" P%d |",proc[sequence[i]].pid);
        fprintf(fout," P%d |",proc[sequence[i]].pid);
    }
    printf("\n0");
    fprintf(fout,"\n0");
    for(int i=0;i<n;i++){
        printf("    %d",proc[sequence[i]].completion_time);
        fprintf(fout,"    %d",proc[sequence[i]].completion_time);
    }
    printf("\n");
    fprintf(fout,"\n");
    fclose(fout);
    printf("\nHasil juga disimpan di file hasil_sjf.txt\n");
    return 0;
}

```

```

3
0 10
1 5
2 8

```

```
root@RIDHO:~# nano sjf_modifikasi.c
root@RIDHO:~# nano input_sjf.txt
root@RIDHO:~# gcc sjf_modifikasi.c -o sjf_modifikasi
root@RIDHO:~# ./sjf_modifikasi
```

=== HASIL PENJADWALAN SJF ===

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13

Average Turnaround Time: 15.00

Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===

Proses dieksekusi dalam urutan: P1 -> P2 -> P3

=== GANTT CHART ===

P1	P2	P3
0	10	15
		23

Hasil juga disimpan di file hasil_sjf.txt

TUGAS 4

Text case 1

```
root@RIDHO:~# nano round_robin.c
root@RIDHO:~# gcc round_robin.c -o round_robin
root@RIDHO:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3
Proses P1
  Arrival Time: 0
  Burst Time: 24

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 3

Proses P4
  Arrival Time: 3
  Burst Time: 6
```

=== PROSES EKSEKUSI ===

Waktu 1: Menjalankan P1 -> Sisa waktu: 21
Waktu 4: Menjalankan P2 -> Selesai pada waktu 7
Waktu 7: Menjalankan P1 -> Sisa waktu: 18
Waktu 10: Menjalankan P3 -> Selesai pada waktu 13
Waktu 13: Menjalankan P4 -> Sisa waktu: 3
Waktu 16: Menjalankan P1 -> Sisa waktu: 15
Waktu 19: Menjalankan P1 -> Sisa waktu: 12
Waktu 22: Menjalankan P1 -> Sisa waktu: 9
Waktu 25: Menjalankan P4 -> Selesai pada waktu 28
Waktu 28: Menjalankan P4 -> Selesai pada waktu 28

=== HASIL PENJADWALAN ROUND ROBIN ===

PID	AT	BT	CT	TAT	WT	
---	--	--	--	---	--	
P1	0	24	793562771		0	0
P2	1	3	7	6	3	
P3	2	3	13	11	8	
P4	3	6	28	25	19	

Average Turnaround Time: 10.50

Average Waiting Time: 7.50

Text case 2

```
root@RIDHO:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4
Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 0
  Burst Time: 5

Proses P3
  Arrival Time: 0
  Burst Time: 8

Proses P4
  Arrival Time: 0
  Burst Time: 12

Proses P5
  Arrival Time: 0
  Burst Time: 6
```


=== PROSES EKSEKUSI ===

Waktu 1: Menjalankan P1 -> Sisa waktu: 6
Waktu 5: Menjalankan P2 -> Sisa waktu: 1
Waktu 9: Menjalankan P3 -> Sisa waktu: 4
Waktu 13: Menjalankan P4 -> Sisa waktu: 8
Waktu 17: Menjalankan P5 -> Sisa waktu: 2
Waktu 21: Menjalankan P1 -> Sisa waktu: 2
Waktu 25: Menjalankan P1 -> Selesai pada waktu 27
Waktu 27: Menjalankan P2 -> Selesai pada waktu 28
Waktu 28: Menjalankan P2 -> Selesai pada waktu 28
Waktu 28: Menjalankan P3 -> Selesai pada waktu 32
Waktu 32: Menjalankan P3 -> Selesai pada waktu 32

=== HASIL PENJADWALAN ROUND ROBIN ===

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	10	27	27	17
P2	0	5	28	28	23
P3	0	8	32	32	24
P4	0	12	32767	-1785014544	32767
P5	0	6	-1927310758	0	0

Average Turnaround Time: -357002880.00

Average Waiting Time: 6566.20

Text case 3

```
root@RIDHO:~# ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2
Proses P1
  Arrival Time: 0
  Burst Time: 8

Proses P2
  Arrival Time: 2
  Burst Time: 4

Proses P3
  Arrival Time: 4
  Burst Time: 9

Proses P4
  Arrival Time: 6
  Burst Time: 5

Proses P5
  Arrival Time: 8
  Burst Time: 3
```

=== PROSES EKSEKUSI ===

Waktu 1: Menjalankan P1 -> Sisa waktu: 6
Waktu 3: Menjalankan P1 -> Sisa waktu: 4
Waktu 5: Menjalankan P2 -> Sisa waktu: 2
Waktu 7: Menjalankan P1 -> Sisa waktu: 2
Waktu 9: Menjalankan P1 -> Selesai pada waktu 11
Waktu 11: Menjalankan P3 -> Sisa waktu: 7
Waktu 13: Menjalankan P1 -> Selesai pada waktu 13
Waktu 13: Menjalankan P2 -> Selesai pada waktu 15
Waktu 15: Menjalankan P4 -> Sisa waktu: 3
Waktu 17: Menjalankan P2 -> Selesai pada waktu 17
Waktu 17: Menjalankan P1 -> Selesai pada waktu 17

=== HASIL PENJADWALAN ROUND ROBIN ===

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	8	17	17	9
P2	2	4	17	15	11
P3	4	9	557891592		32532 878139040
P4	6	5	32764	1765150240	32764
P5	8	3	557888090		0 0

Average Turnaround Time: 353036544.00

Average Waiting Time: 175634352.00

Tugas 5

5. Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

FCFS (First-Come, First-Served) memberikan hasil terbaik (Average Waiting Time /AWT terkecil, mendekati algoritma lain) ketika:

- Semua proses memiliki Burst Time (BT) yang hampir sama. Ketika semua proses memiliki BT yang seragam, urutan eksekusi tidak terlalu memengaruhi waktu tunggu secara keseluruhan.
- Proses dengan BT kecil datang lebih dulu. Jika proses-proses pendek datang pertama, FCFS akan menyelesaikannya dengan cepat, menghasilkan AWT yang rendah.
- Tidak ada Convoy Effect. FCFS paling menderita oleh Convoy Effect, di mana proses dengan BT yang sangat besar (seperti P1: BT=24 di Test Case 1) datang di awal, memaksa semua proses kecil menunggu lama. FCFS akan memberikan hasil terbaik ketika Convoy Effect ini tidak terjadi.

Mengapa SJF hampir selalu memberikan Average WT terkecil?

SJF (Shortest Job First) hampir selalu memberikan Average Waiting Time (AWT) terkecil karena SJF adalah algoritma optimal dalam hal ini.

- Strategi dasarnya adalah menyelesaikan proses yang membutuhkan waktu CPU paling sedikit terlebih dahulu.
- Dengan segera mengeluarkan proses-proses pendek dari antrean, SJF meminimalkan waktu tunggu yang harus ditanggung oleh sejumlah besar proses, sehingga AWT keseluruhan menjadi yang terendah.

2. Context Switching

Algoritma mana yang paling banyak melakukan context switch?

Algoritma Round Robin (RR) adalah yang paling banyak melakukan context switch

- Setiap kali jatah waktu (time quantum) habis atau proses selesai, sistem harus melakukan context switch untuk beralih ke proses berikutnya di antrean Ready.

Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

- Quantum Kecil: $\searrow$ Jumlah Context Switch $\nearrow$ (Tinggi). Jika quantum sangat kecil (misalnya 1), setiap proses akan berganti state setelah setiap unit waktu, menghasilkan overhead yang sangat besar.
- Quantum Besar: $\searrow$ Jumlah Context Switch $\searrow$ (Rendah). Jika quantum sangat besar, RR akan berperilaku mirip dengan FCFS, karena proses yang baru masuk mungkin akan selesai sebelum quantum-nya habis. Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?
- Quantum Kecil (2, 4): Memberikan respon yang sangat cepat (Response Time) karena proses segera mendapatkan CPU. Namun, AWT dan overhead context switch akan tinggi. Untuk Test Case 3 (Quantum=2), context switch sangat intens, namun memastikan semua proses (terutama P2, P4, P5 yang relatif pendek) mendapat jatah eksekusi cepat.
- Quantum Sedang (8): Menghasilkan trade-off yang seimbang. Overhead context switch berkurang, sementara Response Time masih cukup baik. Ini sering menjadi pilihan optimal.
- Quantum Besar (16): Perilaku RR akan mendekati FCFS. Jika quantum lebih besar dari sebagian besar Burst Time, proses-proses tersebut akan selesai dalam satu slice waktu, sehingga context switch hanya terjadi ketika proses selesai, mirip FCFS.

3. Fairness

Algoritma mana yang paling "adil" untuk semua proses?

Algoritma Round Robin (RR) adalah yang paling "adil" (Fair).

- Keadilan di sini didefinisikan sebagai perlakuan yang merata atau pembagian waktu CPU yang sama untuk semua proses, tanpa memandang Arrival Time atau Burst Time.

- RR menjamin bahwa tidak ada proses yang akan "kelaparan" dan setiap proses akan mendapat kesempatan eksekusi dalam waktu yang singkat. Proses mana yang paling dirugikan di algoritma FCFS?

Proses yang paling dirugikan di algoritma FCFS adalah proses dengan Burst Time (BT) kecil yang datang setelah proses dengan BT yang sangat besar.

- Contohnya di Test Case 1, P2, P3, dan P4 dirugikan karena harus menunggu P1 dengan BT=24 selesai terlebih dahulu (meskipun P1 datang paling awal). Proses P2

(BT=3) harus menunggu hingga waktu 24 (CT P1) baru bisa mulai, padahal ia datang di waktu 1.

Apakah ada kemungkinan starvation di SJF?

Ya, ada kemungkinan starvation (kelaparan) di SJF.

- Starvation terjadi ketika proses dengan Burst Time yang sangat panjang terus-menerus terhalang (tidak pernah mendapat CPU) karena selalu ada proses baru dengan Burst Time yang lebih pendek (atau sisa waktu yang lebih pendek) yang tiba di sistem.

- Hal ini sangat mungkin terjadi di sistem dengan beban kerja yang tinggi (banyak proses pendek baru terus datang).

- Solusi untuk mencegah starvation di SJF adalah dengan teknik Aging, yaitu meningkatkan prioritas suatu proses seiring bertambahnya waktu tunggu.

4. Real World Application

Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Round Robin (RR).

- RR memastikan Response Time yang cepat dan keadilan (semua request dilayani secara bergiliran), memberikan pengalaman pengguna yang baik.

Untuk render video (task besar), algoritma mana yang cocok?

Shortest Job First (SJF) atau Prioritas Non-Preemptive.

- Jika kita tahu durasi rendering (BT) dan ingin menyelesaikan render tercepat terlebih dahulu, SJF optimal untuk throughput.
- Namun, untuk tugas yang benar-benar besar seperti video rendering, Prioritas NonPreemptive (di mana tugas rendering diberikan prioritas rendah namun setelah dimulai tidak dapat disela) atau FCFS bisa digunakan jika rendering tersebut adalah satu-satunya tugas yang penting. SJF yang paling logis jika tujuannya adalah meminimalkan waktu penyelesaian tugas pendek-menengah.

Untuk OS desktop (interactive), algoritma mana yang cocok?

Round Robin (RR) dengan Skema Prioritas Dinamis.

- OS desktop (Windows, macOS, Linux) adalah sistem time-sharing yang sangat interaktif.
- RR digunakan sebagai dasar untuk memberikan Response Time yang cepat (misalnya, saat klik mouse atau mengetik).
- Algoritma modern menggabungkan RR dengan Skema Prioritas (prioritas tinggi untuk tugas foreground/interactive dan prioritas rendah untuk tugas background) dan Prioritas Dinamis (prioritas berubah berdasarkan perilaku proses) untuk memberikan fairness dan responsivitas terbaik.

KESIMPULAN

Berdasarkan hasil analisis yang dilakukan, dapat disimpulkan bahwa algoritma SJF memberikan performa terbaik dalam hal efisiensi waktu tunggu rata-rata dibandingkan FCFS dan Round Robin. Algoritma FCFS cocok diterapkan pada sistem sederhana tanpa kebutuhan respons cepat, sementara Round Robin sangat sesuai digunakan pada sistem multitasking yang memerlukan keadilan dan responsivitas tinggi. Nilai quantum pada algoritma Round Robin sangat memengaruhi performa keseluruhan, di mana quantum yang terlalu kecil menyebabkan banyak context switch dan quantum yang terlalu besar membuatnya menyerupai FCFS. Selain itu, kemungkinan starvation pada SJF dapat dihindari dengan menerapkan teknik aging. Dengan memahami karakteristik masing-masing algoritma, pemilihan metode penjadwalan dapat disesuaikan dengan kebutuhan sistem agar diperoleh keseimbangan antara efisiensi, keadilan, dan kecepatan respon.